TRƯỜNG ĐẠI HỌC ĐÔNG Á
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO MÔN HỌC
HỌC PHẦN: THỰC HÀNH DEVOPS

Đề tài:

XÂY DỰNG VÀ TRIỂN KHAI HỆ THỐNG CI/CD TOÀN DIỆN CHO NỀN TẢNG MẠNG XÃ HỘI PINGUP VỚI DOCKER, JENKINS VÀ GITHUB ACTIONS

Giảng viên hướng dẫn: **Ths. Vũ Anh Tuấn**

Sinh viên thực hiện: **Mai Ngọc Phúc**

Mã số sinh viên: 106176

Lớp học phần: IT23M

THÁNG 10 NĂM 2026

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN VỀ DEVOPS VÀ ĐỀ TÀI PINGUP 1
1.1. Bối cảnh và ý nghĩa thực tiễn của văn hóa DevOps 1
1.2. Mục tiêu nghiên cứu và phạm vi bài báo cáo 2
1.3. Giới thiệu tổng quan hệ thống mạng xã hội PingUp 3
1.4. Bảng tổng hợp công nghệ (Tech Stack) sử dụng trong dự án 4
CHƯƠNG 2. THỰC HÀNH NỀN TẢNG: LINUX VÀ BẢO MẬT SSH TRONG DOCKER 6
2.1. Khởi tạo môi trường Ubuntu thực hành trong Docker 6
2.2. Quản trị hệ thống và thực hành các nhóm lệnh Linux cơ bản 7
2.3. Cấu hình dịch vụ SSH Server trong Docker container 10
2.4. Phân tích nguyên nhân và cách khắc phục lỗi Connection Reset 12
2.5. Thiết lập xác thực bảo mật không dùng mật khẩu bằng SSH Key 13
2.6. Tầm quan trọng của SSH Key trong tự động hóa CI/CD 15
CHƯƠNG 3. THIẾT KẾ KIẾN TRÚC HỆ THỐNG VÀ CƠ SỞ DỮ LIỆU 16
3.1. Sơ đồ kiến trúc tổng thể 5 container trên mạng scm-network 16
3.2. Phân tích chi tiết chức năng của các service trong hệ thống 17
3.3. Luồng dữ liệu và giao tiếp liên dịch vụ (REST, Socket, Webhook) 19
3.4. Thiết kế cơ sở dữ liệu MongoDB (7 collections) 20
3.5. Hệ thống Background Jobs và Durable Functions với Inngest 23
CHƯƠNG 4. ĐÓNG GÓI VÀ TỐI ƯU HÓA VỚI DOCKER & DOCKER COMPOSE 26
4.1. Chiến lược Docker Multi-stage Build chuyên sâu 26
4.2. Phân tích Dockerfile Frontend (React Vite → Nginx Runtime) 27
4.3. Phân tích Dockerfile Backend và tối ưu hóa npm ci 29
4.4. Quản lý đa dịch vụ với Docker Compose và cơ chế Healthcheck 31
4.5. Phân tích tầm quan trọng của cờ init: true chống zombie process 33
4.6. Cấu hình triển khai Production với docker-compose.prod.yml 34
CHƯƠNG 5. XÂY DỰNG PIPELINE CI/CD TỰ ĐỘNG VỚI JENKINS 36
5.1. Mô hình Docker-out-of-Docker (DooD) và phân tích so sánh với DinD 36
5.2. Cấu hình môi trường Jenkins và bảo mật Credentials GHCR 38
5.3. Phân tích chi tiết Declarative Jenkinsfile qua từng giai đoạn 39
5.4. Quy trình Deploy không gián đoạn và rollback bằng Git SHA 43
CHƯƠNG 6. TỰ ĐỘNG HÓA CI TRÊN CLOUD VỚI GITHUB ACTIONS 45
6.1. Kiến trúc workflow GitHub Actions (.github/workflows) 45
6.2. Phân tích 6 jobs: Kiểm thử, Kiểm tra build và Đẩy Image 46
6.3. Kỹ thuật tối ưu: Path Filtering, GHA Cache và GITHUB_TOKEN 48
6.4. Bảng so sánh toàn diện giữa Jenkins và GitHub Actions 50

CHƯƠNG 7. HƯỚNG DẪN CÀI ĐẶT, VẬN HÀNH VÀ KỊCH BẢN KIỂM THỬ (DEMO) 52

7.1. Yêu cầu môi trường phần cứng và phần mềm máy trạm 52

7.2. Hướng dẫn từng bước khởi chạy toàn bộ hệ thống từ mã nguồn 53

7.3. Kịch bản kiểm thử các dịch vụ thực tế và đường dẫn truy cập 55

7.4. Hướng dẫn xử lý các sự cố thường gặp (Troubleshooting Guide) 57

TÀI LIỆU THAM KHẢO 58

DANH MỤC HÌNH ẢNH

Hình 2.1	Khởi tạo container ubuntu-lab từ image ubuntu:latest và chạy apt-get update	6
Hình 2.2	Cài đặt các công cụ quản trị hệ thống bằng apt-get install	7
Hình 2.3	Kết quả thực thi các lệnh kiểm tra hệ thống và thao tác tệp tin	8
Hình 2.4	Kết quả thực thi lệnh quản lý người dùng, phân quyền và giám sát tiến trình	9
Hình 2.5	Kiểm tra cấu hình địa chỉ IP, kết nối mạng và các cổng lắng nghe	10
Hình 2.6	Sơ đồ nguyên lý xác thực SSH Key vào Docker container qua cổng 2222	11
Hình 2.7	Tạo container ubuntu-ssh với port forwarding 2222:22 và cấu hình openssh-server	12
Hình 2.8	Hiện tượng lỗi Connection reset by peer và cách xử lý chạy foreground sshd -D	13
Hình 2.9	Khởi tạo cặp khóa xác thực RSA 4096-bit bằng công cụ ssh-keygen	14
Hình 2.10	Sao chép khóa công khai vào container với lệnh ssh-copy-id thành công	15
Hình 3.1	Sơ đồ kiến trúc tổng thể hệ thống PingUp với 5 containers và luồng CI/CD	17
Hình 3.2	Thiết kế cấu trúc Schema MongoDB với 7 collections và quan hệ liên kết	21

Hình 4.1 Sơ đồ chiến lược Docker Multi-stage Build cho Frontend và Backend 27

Hình 5.1 Sơ đồ quy trình thực thi Pipeline CI/CD 4 giai đoạn trên Jenkins 40

Hình 6.1 Kiến trúc các Jobs chạy song song trong GitHub Actions workflow 46

DANH MỤC BẢNG BIỂU

Bảng 1.1 Bảng tổng hợp công nghệ (Tech Stack) và phiên bản sử dụng trong dự án 4

Bảng 2.1 Tổng hợp các nhóm lệnh Linux cơ bản đã thực hành trên môi trường container 10

Bảng 3.1 Danh sách các container dịch vụ, image base và cổng kết nối hệ thống 18

Bảng 3.2 Chi tiết 6 Background Functions được đăng ký và thực thi trên Inngest 24

Bảng 4.1 So sánh đặc điểm kỹ thuật giữa hai Dockerfile trong hệ thống 30

Bảng 5.1 Phân tích so sánh giữa kiến trúc Docker-out-of-Docker (DooD) và DinD 37

Bảng 6.1 So sánh đối chiếu toàn diện giữa Jenkins Pipeline và GitHub Actions 50

Bảng 7.1 Bảng danh mục các cổng dịch vụ và đường dẫn kiểm thử chức năng 55

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

CI/CD	Continuous Integration / Continuous Deployment (Tích hợp và Triển khai liên tục)
DooD	Docker-out-of-Docker (Kỹ thuật chia sẻ Docker socket từ Host vào Container)
DinD	Docker-in-Docker (Kỹ thuật chạy một Docker daemon độc lập bên trong Container)
GHCR	GitHub Container Registry (Hệ thống lưu trữ và quản lý Docker image của GitHub)
SPA	Single Page Application (Ứng dụng web một trang được render phía client)
ODM	Object Document Mapping (Ánh xạ đối tượng sang tài liệu CSDL phi quan hệ)
JWT	JSON Web Token (Tiêu chuẩn mở truyền tải thông tin an toàn dưới dạng JSON)
PAT	Personal Access Token (Khóa truy cập cá nhân dùng cho xác thực bảo mật)
CORS	Cross-Origin Resource Sharing (Cơ chế bảo mật chia sẻ tài nguyên giữa các domain)
PID 1	Process Identifier 1 (Tiến trình gốc đầu tiên được khởi chạy trong hệ điều hành/container)

CHƯƠNG 1. TỔNG QUAN VỀ DEVOPS VÀ ĐỀ TÀI PINGUP

1.1. Bối cảnh và ý nghĩa thực tiễn của văn hóa DevOps

Trong quy trình phát triển phần mềm truyền thống, sự phân tách biệt lập giữa đội ngũ phát triển (Development) và đội ngũ vận hành (Operations) thường tạo ra 'bức tường ngăn cách' (wall of confusion). Đội ngũ phát triển tập trung vào việc liên tục bổ sung các tính năng mới, trong khi đội ngũ vận hành ưu tiên hàng đầu sự ổn định và tính sẵn sàng của hạ tầng sản xuất. Sự xung đột về mục tiêu này thường dẫn đến các chu kỳ phát hành kéo dài, quy trình bàn giao thủ công dễ phát sinh sai sót, và đặc biệt là hiện tượng ứng dụng hoạt động tốt trên máy phát triển cục bộ nhưng lại gặp lỗi nghiêm trọng khi triển khai trên môi trường máy chủ.

DevOps ra đời như một sự kết hợp chặt chẽ giữa văn hóa làm việc, quy trình chuẩn hóa và tập hợp các công cụ tự động hóa, nhằm rút ngắn khoảng cách giữa hai bộ phận. Thông qua việc áp dụng tự động hóa vào toàn bộ chu kỳ phát triển phần mềm (từ khâu kiểm thử mã nguồn, đóng gói ứng dụng, phân phối hình ảnh container cho đến triển khai tự động lên hệ thống đích), DevOps giúp các tổ chức công nghệ tăng tốc độ phân phối sản phẩm, đảm bảo tính nhất quán tuyệt đối giữa các môi trường, và nâng cao độ tin cậy của toàn bộ hệ thống dịch vụ.

1.2. Mục tiêu nghiên cứu và phạm vi bài báo cáo

Trong khuôn khổ học phần Thực hành DevOps tại Trường Đại học Đông Á, dưới sự hướng dẫn trực tiếp của Ths. Vũ Anh Tuấn, bài báo cáo này được thực hiện với các mục tiêu trọng tâm sau đây:

- **Thực hành nền tảng:**Nghiên cứu và làm chủ các kỹ năng quản trị hệ điều hành Linux cơ bản, thao tác dòng lệnh, quản lý tài nguyên, tiến trình và phân quyền trên môi trường Docker container.
- **Bảo mật truy cập từ xa:**Khởi tạo và cấu hình dịch vụ OpenSSH Server trong container, xử lý các sự cố ngắt kết nối thực tế và thiết lập cơ chế xác thực an toàn không dùng mật khẩu thông qua cặp khóa SSH Key (RSA 4096-bit).
- **Thiết kế kiến trúc hệ thống:**Phân tích kiến trúc hệ thống mạng xã hội thời gian thực đa dịch vụ PingUp, thiết kế cấu trúc dữ liệu MongoDB và cơ chế thực thi Background Jobs tin cậy với Inngest.
- **Đóng gói container hóa:**Xây dựng chiến lược Docker Multi-stage Build tối ưu hóa kích thước image, cô lập môi trường biên dịch và vận hành; điều phối cụm 5 containers đồng bộ bằng Docker Compose với cơ chế Healthcheck và quản lý tiến trình init.
- **Tự động hóa CI/CD:**Xây dựng và tối ưu hóa hai luồng Pipeline tự động hóa: Jenkins (theo mô hình Docker-out-of-Docker triển khai trực tiếp trên máy chủ) và GitHub Actions (kiểm thử và đóng gói tự động trên nền tảng đám mây GitHub Container Registry - GHCR).
- **Kiểm thử và đánh giá:**Xây dựng kịch bản kiểm thử, hướng dẫn vận hành chi tiết và tổng hợp bộ câu hỏi phản biện kỹ thuật phục vụ buổi vấn đáp trực tiếp tại lớp học.

1.3. Giới thiệu tổng quan hệ thống mạng xã hội PingUp

Dự án được lựa chọn để thực hành triển khai DevOps là nền tảng mạng xã hội thời gian thực PingUp. Đây là một ứng dụng web hiện đại, có độ phức tạp cao, tích hợp đầy đủ các luồng nghiệp vụ thực tế của một nền tảng truyền thông xã hội:

- **Phân hệ xác thực:**Xác thực người dùng an toàn tích hợp Clerk Authentication với cơ chế JIT (Just-In-Time) Auto-Provisioning đồng bộ tài khoản tự động vào cơ sở dữ liệu MongoDB.
- **Phân hệ bài viết và tương tác:**Người dùng có thể tạo bài viết định dạng văn bản hoặc đa phương tiện (hình ảnh được tối ưu hóa qua ImageKit CDN), tương tác thích (like), chia sẻ bài viết và bình luận lồng nhau đa cấp.
- **Phân hệ Story 24h tự động:**Cho phép người dùng đăng tải các khoảnh khắc ngắn (Story) với thời gian tồn tại đúng 24 giờ. Hệ thống tự động thu hồi và xóa dữ liệu sau 24 giờ nhờ tác vụ nền Durable Function của Inngest mà không duy trì kết nối liên tục.
- **Phân hệ tin nhắn thời gian thực:**Xây dựng trên nền tảng Socket.IO, hỗ trợ phòng chat cá nhân, thông báo trạng thái đang gõ phím và truyền nhận tin nhắn tức thì.
- **Phân hệ thông báo và Email:**Hệ thống thông báo đẩy 11 trạng thái sự kiện khác nhau và dịch vụ gửi thư điện tử tự động qua giao thức SMTP Brevo/Nodemailer nhắc nhở kết nối hoặc tin nhắn chưa đọc.

1.4. Bảng tổng hợp công nghệ (Tech Stack) sử dụng trong dự án

Bảng 1.1 dưới đây tổng hợp chi tiết các công nghệ, thư viện và phiên bản chính được áp dụng xuyên suốt hệ thống dự án PingUp:

Thành phần hệ thống	Công nghệ sử dụng	Phiên bản	Vai trò và nhiệm vụ kỹ thuật
Giao diện người dùng	React + Vite	React 19 / Vite 7	Xây dựng Single Page Application (SPA), render UI tốc độ cao
Định kiểu & Hiệu ứng	Tailwind CSS v4 + Framer Motion	v4.1	Thiết kế giao diện hiện đại, tối ưu CSS bundle và hiệu ứng mượt
Quản lý trạng thái	Redux Toolkit	RTK 2.8	Quản lý trạng thái đăng nhập, dữ liệu người dùng và danh sách bài viết
Giao tiếp thời gian thực	Socket.IO Client / Server	v4.8.1	Thiết lập kết nối song công WebSockets cho tin nhắn và thông báo

Máy chủ dịch vụ API	Node.js + Express.js	Node 22 / Express 5	Xử lý logic nghiệp vụ, cung cấp RESTful APIs và tiếp nhận Webhooks
Cơ sở dữ liệu chính	MongoDB Community Server	v7.0	Lưu trữ dữ liệu phi quan hệ với cấu trúc JSON-like linh hoạt
Object Document Mapper	Mongoose	v8.17	Định nghĩa Schema, kiểm tra ràng buộc kiểu dữ liệu và quan hệ reference
Hệ thống tác vụ nền	Inngest Server & SDK	v3.54	Thực thi tác vụ nền bền bỉ, hàng đợi sự kiện và cron job định kỳ
Dịch vụ xác thực	Clerk Identity Platform	v1.7 Express	Quản lý phiên đăng nhập, cấp phát JWT và tích hợp Webhook đồng bộ
Lưu trữ đa phương tiện	ImageKit CDN	v6.0 SDK	Xử lý tối ưu hóa dung lượng hình ảnh, CDN phân phối băng thông cao
Máy chủ Web tĩnh	Nginx Alpine	v1.27	Phục vụ bundle React tĩnh, định tuyến SPA fallback và nén dữ liệu Gzip
Môi trường đóng gói	Docker & Docker Compose	Docker 29 / Compose 5	Đóng gói toàn diện môi trường và điều phối 5 containers nội bộ
Tự động hóa CI/CD	Jenkins (DooD) & GitHub Actions	Jenkins LTS / Actions v4	Tự động hóa chu trình kiểm thử, build multi-arch và triển khai
Hệ thống lưu trữ Image	GitHub Container Registry (GHCR)	ghcr.io	Lưu trữ và phân phối Docker image đóng gói đa kiến trúc AMD64/ARM64

CHƯƠNG 2. THỰC HÀNH NỀN TẢNG: LINUX VÀ BẢO MẬT SSH TRONG DOCKER

2.1. Khởi tạo môi trường Ubuntu thực hành trong Docker

Hệ điều hành Linux là nền tảng cốt lõi của hạ tầng DevOps hiện đại, nơi mà phần lớn các container, máy chủ biên và các công cụ điều phối (Kubernetes, Docker Swarm) đều vận hành trên nhân Linux. Nhằm trang bị kỹ năng thao tác dòng lệnh chuyên sâu, bước đầu tiên trong chuỗi thực hành của môn học là khởi tạo một container chạy hệ điều hành Ubuntu độc lập từ image chính thức.

Sinh viên tiến hành kéo image Ubuntu mới nhất và khởi tạo container mang tên 'ubuntu-lab' ở chế độ tương tác nền bằng câu lệnh:

```
docker run -it -d --name ubuntu-lab ubuntu:latest
docker exec -it ubuntu-lab bash
apt-get update
```

Lệnh trên khởi chạy container Ubuntu ở chế độ daemon (-d) và mở phiên làm việc tương tác (-it). Ngay sau khi truy cập vào container thông qua lệnh docker exec, hệ thống được cập nhật danh mục gói phần mềm thông qua trình quản lý apt-get.

```
Run 'docker run --help' for more information
ngocphucit23m@LeviNgyx:~$ docker run -it -d --name ubuntu-lab ubuntu:latest
8d51496d7d34129f4b0fa02da8ff3c3ed51580372eb107358480372832cb941a
ngocphucit23m@LeviNgyx:~$ docker exec -it ubuntu-lab bash
root@8d51496d7d34:/# docker exec -it ubuntu-lab bash
bash: docker: command not found
root@8d51496d7d34:/# exit
exit
ngocphucit23m@LeviNgyx:~$ docker exec -it ubuntu-lab bash
root@8d51496d7d34:/# apt-get update
Get:1 http://archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/multiverse amd64 Packages [24.6 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [252 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [739 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute/universe amd64 Packages [20.1 MB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [609 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute/main amd64 Packages [1874 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute/restricted amd64 Packages [189 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute/multiverse amd64 Packages [352 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [26.1 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [625 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [911 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [417 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Packages [3308 B]
Fetched 26.7 MB in 6s (4602 kB/s)
Reading package lists... Done
root@8d51496d7d34:/#
```

Hình 2.1: Khởi tạo container ubuntu-lab từ image ubuntu:latest và thực thi cập nhật chỉ mục apt-get update

2.2. Quản trị hệ thống và thực hành các nhóm lệnh Linux cơ bản

Sau khi danh mục gói được làm mới, các công cụ quản trị hệ thống và mạng thiết yếu được tiến hành cài đặt vào container thông qua lệnh apt-get install bao gồm: sudo, curl, net-tools (cung cấp lệnh ifconfig, netstat), iputils-ping (kiểm tra kết nối mạng), và trình biên tập văn bản nano/vim.

```
root@8d51496d7d34:/#  apt-get install -y sudo vim net-tools iputils-ping openssh-server curl wget
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following additional packages will be installed:
  adduser bash-completion ca-certificates dbus dbus-bin dbus-daemon dbus-session-bus-common dbus-system-bus-common
  dbus-user-session dmsetup krb5-locales libapparmor1 libbrotli1 libcap2 libcbor0.10 libcryptsetup12 libcurl4t64
  libdbus-1-3 libdevmapper1.02.1 libedit2 libexpat1 libffi8 libfido2-1 libgnutls30t64 libgpm2 libgssapi-krb5-2
  libhogweed6t64 libidn2-0 libjson-c5 libk5crypto3 libkeyutils1 libkmod2 libkrb5-3 libkrb5support0 libldap-common
  libldap2 libnettle8t64 libnghttp2-14 libnss-systemd libp11-kit0 libpam-systemd libpsl5t64 libpython3-stdlib
  libpython3.14 libpython3.14-minimal libpython3.14-stdlib libreadline8t64 librtmp1 libsasl2-2 libsasl2-modules
  libsasl2-modules-db libsodium23 libsqlite3-0 libssh2-1t64 libsystemd-shared libtasn1-6 libtext-charwidth-perl
  libtext-wrapi18n-perl libunistring5 libwrap0 libx11-6 libx11-data libxau6 libxcb1 libxdmcp6 libxext6 libxmuu1
  linux-sysctl-defaults lsb-release media-types ncurses-term netbase openssh-client openssh-sftp-server openssl
  publicsuffix python3 python3-autocommand python3-bcrypt python3-blinker python3-cffi-backend python3-cryptography
  python3-distro python3-httplib2 python3-inflect python3-jaraco.context python3-jaraco.functools python3-jaraco.text
  python3-jwt python3-launchpadlib python3-lazr.restfulclient python3-lazr.uri python3-minimal python3-more-itertools
  python3-oauthlib python3-pkg-resources python3-pyparsing python3-setuptools python3-typeguard
  python3-typing-extensions python3-wadllib python3-zipp python3.14 python3.14-minimal readline-common ssh-import-id
  sudo-common sudo-rs systemd systemd-cryptsetup systemd-resolved systemd-sysv systemd-timesyncd tzdata ucf vim-common
  vim-runtime xauth xxd
Suggested packages:
  liblocale-gettext-perl perl cron quota ecryptfs-utils gnutls-bin gpm krb5-doc krb5-user libtss2-esys-3.0.2-0t64
  libtss2-mu-4.0.1-0t64 libtss2-rc0t64 systemd-userdbd libsasl2-modules-gssapi-mit | libsasl2-modules-gssapi-heimdal
  libsasl2-modules-ldap libsasl2-modules-otp libsasl2-modules-sql libarchive13t64 libbpf1 libdw1t64 libelf1t64
  libpwquality1 libqrencode4 keychain libpam-ssh monkeysphere ssh-askpass molly-guard ufw python3-doc python3-tk
  python3-venv python-blinker-doc python-cryptography-doc python3-cryptography-vectors python-jwt-doc python3-crypto
  python3-keyring python3-testresources python-pyparsing-doc python-setuptools-doc python3.14-venv python3.14-doc
  binutils binfmt-support readline-doc systemd-container systemd-homed systemd-boot systemd-repart
  libtss2-tcti-device0 polkitd ctags vim-doc vim-scripts
The following NEW packages will be installed:
  adduser bash-completion ca-certificates curl dbus dbus-bin dbus-daemon dbus-session-bus-common
  dbus-system-bus-common dbus-user-session dmsetup iputils-ping krb5-locales libapparmor1 libbrotli1 libcap2
```

Hình 2.2: Quá trình cài đặt bổ sung các gói tiện ích quản trị hệ thống bằng apt-get install

2.2.1. Nhóm lệnh kiểm tra hệ thống và thao tác tệp tin

Trong môi trường dòng lệnh Linux, việc kiểm tra thông số kiến trúc hạt nhân và phân cấp cây thư mục được thực hiện thông qua các lệnh tiêu chuẩn:

- **Kiểm tra nhân:**uname -a: Trả về thông tin chi tiết về phiên bản nhân Linux (Kernel version), kiến trúc phần cứng (x86_64) và tên hostname.
- **Xác định vị trí:**pwd: Xác định đường dẫn thư mục làm việc hiện tại (Print Working Directory).
- **Liệt kê chi tiết:**ls -la: Liệt kê toàn bộ các tệp tin và thư mục bao gồm cả các tệp ẩn bắt đầu bằng dấu chấm, kèm theo chi tiết quyền truy cập, chủ sở hữu, dung lượng và thời gian sửa đổi cuối.
- **Thao tác dữ liệu:**mkdir, touch, cp, mv, rm: Bộ lệnh tạo thư mục mới, tạo tệp rỗng, sao chép dữ liệu, di chuyển/đổi tên và xóa tệp tin.

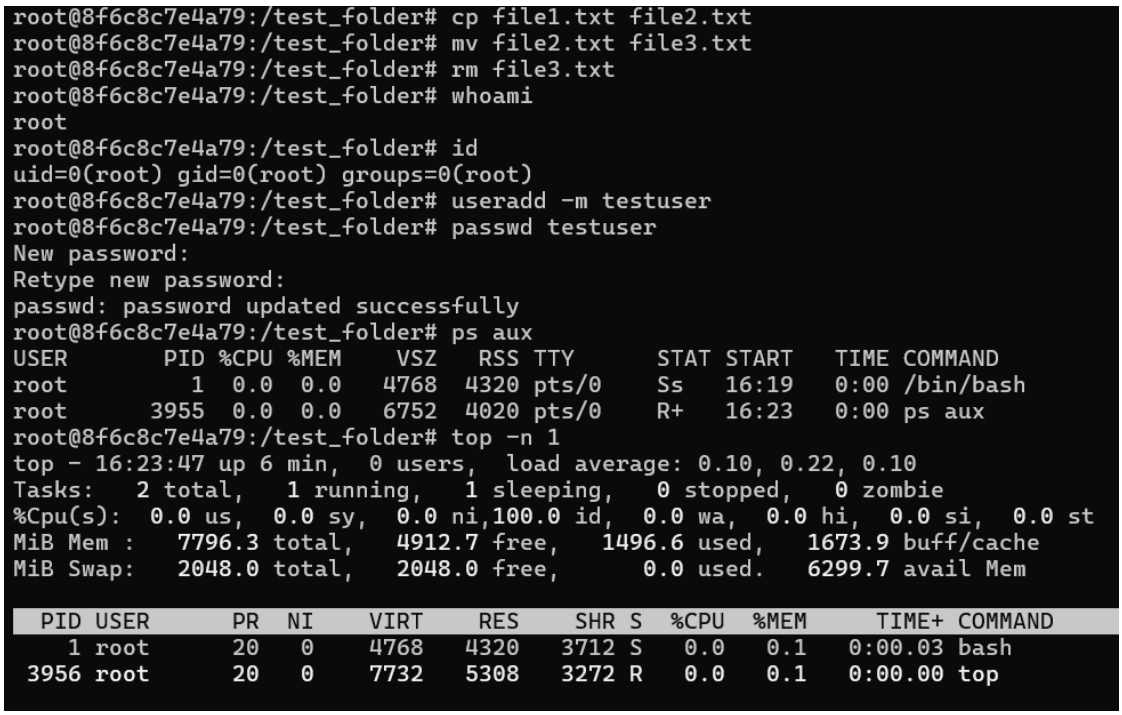
```
root@8f6c8c7e4a79:/# uname -a
Linux 8f6c8c7e4a79 5.15.167.4-microsoft-standard-WSL2 #1 SMP Tue Nov 5 00:21:55 UTC 2024 x86_64 GNU/Linux
root@8f6c8c7e4a79:/# lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 26.04 LTS
Release:        26.04
Codename:       resolute
root@8f6c8c7e4a79:/# pwd
/
root@8f6c8c7e4a79:/# ls -la
total 76
drwxr-xr-x   1 root root 4096 Sep  4 16:19 .
drwxr-xr-x   1 root root 4096 Sep  4 16:19 ..
-rwxr-xr-x   1 root root    0 Sep  4 16:19 .dockerenv
drwxr-xr-x   2 root root 4096 Aug 17 16:02 .rock
lrwxrwxrwx   1 root root    7 Apr 20 15:46 bin -> usr/bin
drwxr-xr-x   2 root root 4096 Apr 20 15:46 boot
drwxr-xr-x   5 root root  360 Sep  4 16:19 dev
drwxr-xr-x   1 root root 4096 Sep  4 16:21 etc
drwxr-xr-x   3 root root 4096 Aug 17 16:02 home
lrwxrwxrwx   1 root root    7 Apr 20 15:46 lib -> usr/lib
lrwxrwxrwx   1 root root    9 Apr 20 15:46 lib64 -> usr/lib64
drwxr-xr-x   2 root root 4096 Aug 17 16:01 media
drwxr-xr-x   2 root root 4096 Aug 17 16:01 mnt
drwxr-xr-x   2 root root 4096 Aug 17 16:01 opt
dr-xr-xr-x 300 root root    0 Sep  4 16:19 proc
drwx------   1 root root 4096 Sep  4 16:21 root
drwxr-xr-x   1 root root 4096 Sep  4 16:21 run
lrwxrwxrwx   1 root root    8 Apr 20 15:46 sbin -> usr/sbin
drwxr-xr-x   2 root root 4096 Aug 17 16:01 srv
dr-xr-xr-x  11 root root    0 Sep  4 16:19 sys
drwxrwxrwt   1 root root 4096 Sep  4 16:21 tmp
drwxr-xr-x   1 root root 4096 Aug 17 16:00 usr
drwxr-xr-x   1 root root 4096 Aug 17 16:02 var
root@8f6c8c7e4a79:/# mkdir test_folder
root@8f6c8c7e4a79:/# cd test_folder
root@8f6c8c7e4a79:/test_folder# touch file1.txt file2.txt
root@8f6c8c7e4a79:/test_folder# echo "Hello Linux" > file1.txt
root@8f6c8c7e4a79:/test_folder# cat file1.txt
Hello Linux
```

Hình 2.3: Kết quả thực thi các lệnh uname, pwd, ls -la và các thao tác quản lý tệp tin trên console

2.2.2. Nhóm lệnh quản lý người dùng, phân quyền và tiến trình

Bảo mật trong DevOps đòi hỏi việc hạn chế sử dụng trực tiếp tài khoản root và tuân thủ nguyên tắc đặc quyền tối thiểu (Least Privilege). Sinh viên thực hành tạo mới tài khoản người dùng, thiết lập mật khẩu và phân quyền quản trị:

- **Tạo người dùng:**useradd -m -s /bin/bash devuser: Khởi tạo người dùng mới tên devuser kèm thư mục cá nhân /home/devuser.
- **Đặt mật khẩu:**passwd devuser: Thiết lập mật khẩu xác thực cho người dùng mới tạo.
- **Kiểm tra định danh:**whoami và id: Kiểm tra định danh người dùng hiện tại, mã định danh người dùng (UID) và mã định danh nhóm (GID).
- **Giám sát tiến trình:**ps aux: Liệt kê toàn bộ các tiến trình đang hoạt động trong hệ thống kèm lượng tiêu thụ CPU và bộ nhớ RAM.
- **Giám sát động:**top: Trình theo dõi tiến trình và tài nguyên hệ thống theo thời gian thực tương tác.



Hình 2.4: Kết quả thực thi lệnh quản lý người dùng (useradd, passwd, id) và giám sát tiến trình hệ thống (ps aux)

2.2.3. Nhóm lệnh quản trị mạng và cổng kết nối

Container trong Docker được cấp phát một địa chỉ IP nội bộ thuộc mạng bridge ảo. Sinh viên tiến hành kiểm tra giao diện mạng và khả năng thông suốt kết nối:

- **Kiểm tra giao diện:**ifconfig -a: Hiển thị các giao diện mạng mạng (giao diện eth0 với địa chỉ IP nội bộ cấp phát và giao diện loopback lo).
- **Kiểm tra kết nối:**ping -c 4 google.com: Kiểm tra khả năng phân giải tên miền DNS và kết nối định tuyến ra mạng Internet từ bên trong container.
- **Kiểm tra cổng dịch vụ:**netstat -tuln: Liệt kê toàn bộ các cổng TCP/UDP đang ở trạng thái lắng nghe (LISTEN) cùng thông tin địa chỉ socket.

```
root@8f6c8c7e4a79:/test_folder# ifconfig -a
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 172.17.0.2  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 9e:ec:f1:f0:a7:81  txqueuelen 0  (Ethernet)
        RX packets 105838  bytes 145778496 (145.7 MB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 95996  bytes 6988445 (6.9 MB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

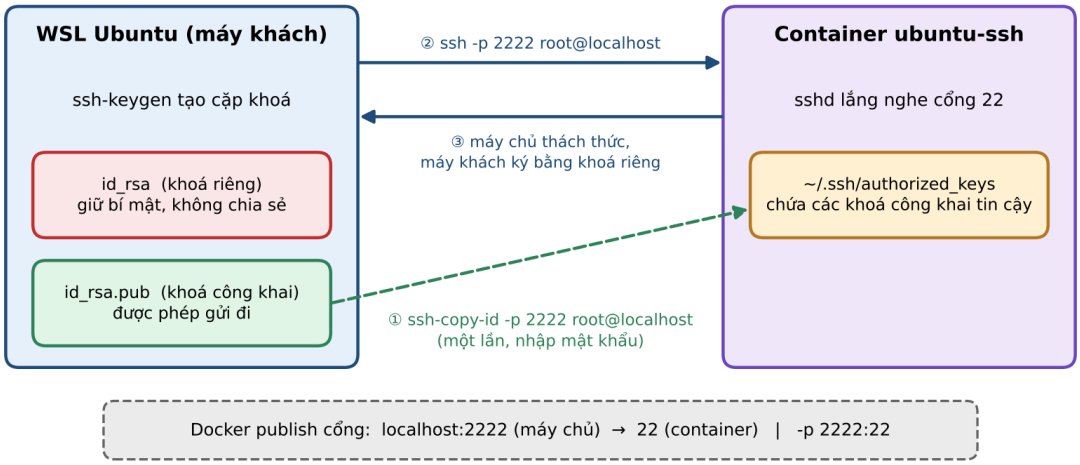
root@8f6c8c7e4a79:/test_folder# ping -c 4 google.com
PING google.com (142.250.71.174) 56(84) bytes of data.
64 bytes from 142.250.71.174: icmp_seq=1 ttl=63 time=68.1 ms
64 bytes from 142.250.71.174: icmp_seq=2 ttl=63 time=153 ms
64 bytes from 142.250.71.174: icmp_seq=3 ttl=63 time=61.8 ms
64 bytes from 142.250.71.174: icmp_seq=4 ttl=63 time=63.7 ms

--- google.com ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3006ms
rtt min/avg/max/mdev = 61.790/86.587/152.752/38.267 ms
root@8f6c8c7e4a79:/test_folder# netstat -tulpn
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
root@8f6c8c7e4a79:/test_folder# apt-get update
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Hit:2 http://security.ubuntu.com/ubuntu resolute-security InRelease
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Hit:4 http://archive.ubuntu.com/ubuntu resolute-backports InRelease
Get:5 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [338 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [743 kB]
Fetched 1219 kB in 17s (71.5 kB/s)
```

Hình 2.5: Kiểm tra cấu hình địa chỉ IP qua ifconfig, kiểm tra định tuyến mạng qua ping và cổng lắng nghe qua netstat

2.3. Cấu hình dịch vụ SSH Server trong Docker container

Giao thức SSH (Secure Shell) là chuẩn mực bảo mật cho việc quản trị máy chủ từ xa và là phương thức kết nối then chốt để các máy chủ tự động hóa (như Jenkins) có thể kết nối điều khiển môi trường triển khai. Để mô phỏng môi trường máy chủ từ xa, sinh viên thực hiện cấu hình dịch vụ OpenSSH Server bên trong một container Docker chuyên dụng có ánh xạ cổng mạng.



Hình 2.6: Sơ đồ nguyên lý xác thực SSH Key từ máy phát triển vào Docker container thông qua cổng chuyển tiếp 2222

Quá trình triển khai bao gồm việc tạo container 'ubuntu-ssh' với thiết lập cổng ánh xạ chuyển tiếp từ cổng 2222 trên máy chủ Host vào cổng tiêu chuẩn 22 của SSH daemon bên trong container:

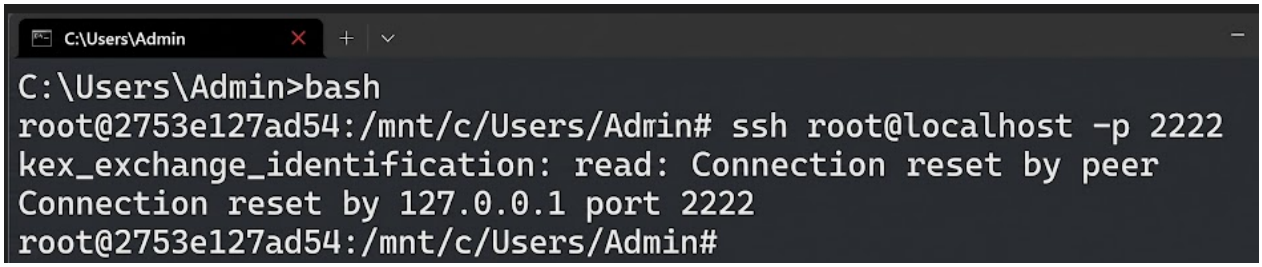
```
docker run -d --name ubuntu-ssh -p 2222:22 ubuntu:latest sleep infinity
docker exec -it ubuntu-ssh bash
apt-get update && apt-get install -y openssh-server
mkdir /var/run/sshd
echo 'root:Devops@2026' | chpasswd
```

```
ngocphucit23m@LeviNgyx:~$ docker run -d --name ubuntu-ssh -p 2222:22 ubuntu:latest sleep infinity
c7526b4577ef6a4139cfd7d4155ba060c13657aafacc28b0f1cbb7a40db25c81
ngocphucit23m@LeviNgyx:~$ docker exec -it ubuntu-ssh bash
root@c7526b4577ef:/# apt-get update && apt-get install -y openssh-server
Get:1 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:2 http://archive.ubuntu.com/ubuntu resolute InRelease [136 kB]
Get:3 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [252 kB]
Get:4 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:5 http://security.ubuntu.com/ubuntu resolute-security/restricted amd64 Packages [609 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [739 kB]
Get:8 http://archive.ubuntu.com/ubuntu resolute/restricted amd64 Packages [189 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/multiverse amd64 Packages [24.6 kB]
Get:10 http://archive.ubuntu.com/ubuntu resolute/multiverse amd64 Packages [352 kB]
Get:11 http://archive.ubuntu.com/ubuntu resolute/main amd64 Packages [1874 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute/universe amd64 Packages [20.1 MB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/restricted amd64 Packages [625 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [417 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [911 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/multiverse amd64 Packages [26.1 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Packages [3308 B]
Fetched 26.7 MB in 6s (4468 kB/s)
Reading package lists... Done
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Solving dependencies... Done
The following additional packages will be installed:
  adduser ca-certificates dbus dbus-bin dbus-daemon dbus-session-bus-common dbus-system-bus-common dbus-user-session dmsetup
  krb5-locales libapparmor1 libcbor0.10 libcryptsetup12 libdbus-1-3 libdevmapper1.02.1 libedit2 libexpat1 libffi8 libfido2-1
```

Hình 2.7: Cài đặt gói openssh-server, tạo thư mục vận hành /var/run/sshd và thiết lập mật khẩu truy cập

2.4. Phân tích nguyên nhân và cách khắc phục lỗi Connection Reset

Trong quá trình thực nghiệm kết nối lần đầu qua lệnh ssh root@localhost -p 2222, sinh viên ghi nhận lỗi thông báo: 'kex_exchange_identification: read: Connection reset by peer' hoặc phiên làm việc lập tức bị ngắt kết nối.

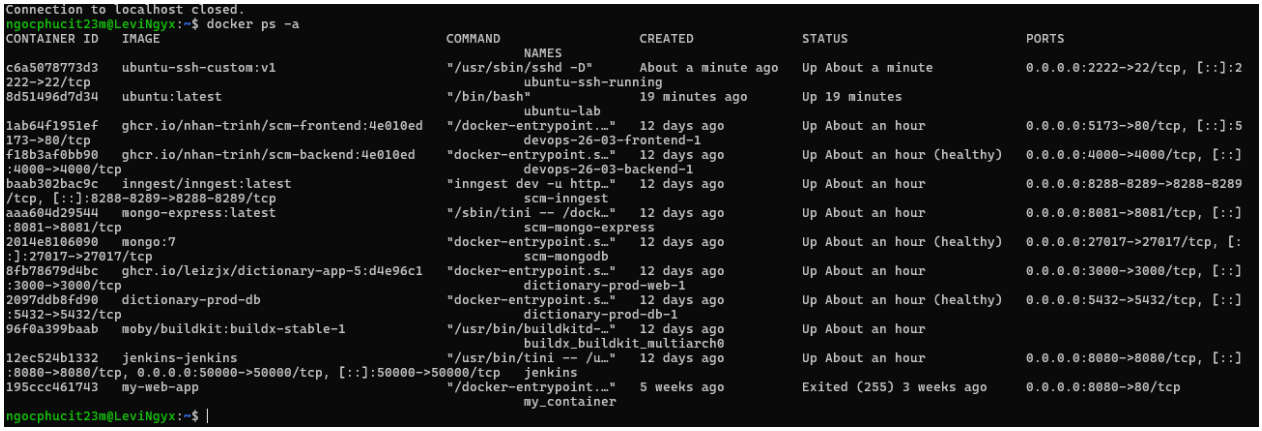


Hình 2.8: Thông báo lỗi Connection reset by peer khi tiến hành kết nối SSH vào container lần đầu

Phân tích nguyên nhân kỹ thuật và giải pháp xử lý:
Trong mô hình Docker, một container chỉ tồn tại khi tiến trình chính (PID 1) của nó còn hoạt động. Khi khởi động dịch vụ SSH bằng lệnh 'service ssh start', dịch vụ này tự động chạy ngầm dưới dạng background daemon và trả quyền điều khiển về shell, khiến tiến trình chính hoàn tất và cổng kết nối bị đóng lại lập tức.
Giải pháp khắc phục chuẩn mực trong DevOps: Cấu hình tiến trình daemon OpenSSH chạy trực tiếp ở chế độ foreground (không tách tiến trình) bằng cờ '-D' (/usr/sbin/sshd -D). Khi đó sshd sẽ giữ vai trò là PID 1, duy trì container luôn hoạt động và sẵn sàng tiếp nhận các kết nối mạng.

Sinh viên đóng gói cấu hình đã sửa đổi thành image mới 'ubuntu-ssh-custom:v1' và khởi chạy container với lệnh chỉ định rõ ràng:

```
docker commit ubuntu-ssh ubuntu-ssh-custom:v1
docker run -d --name ubuntu-ssh-running -p 2222:22 ubuntu-ssh-custom:v1 /usr/sbin/sshd
-D
```



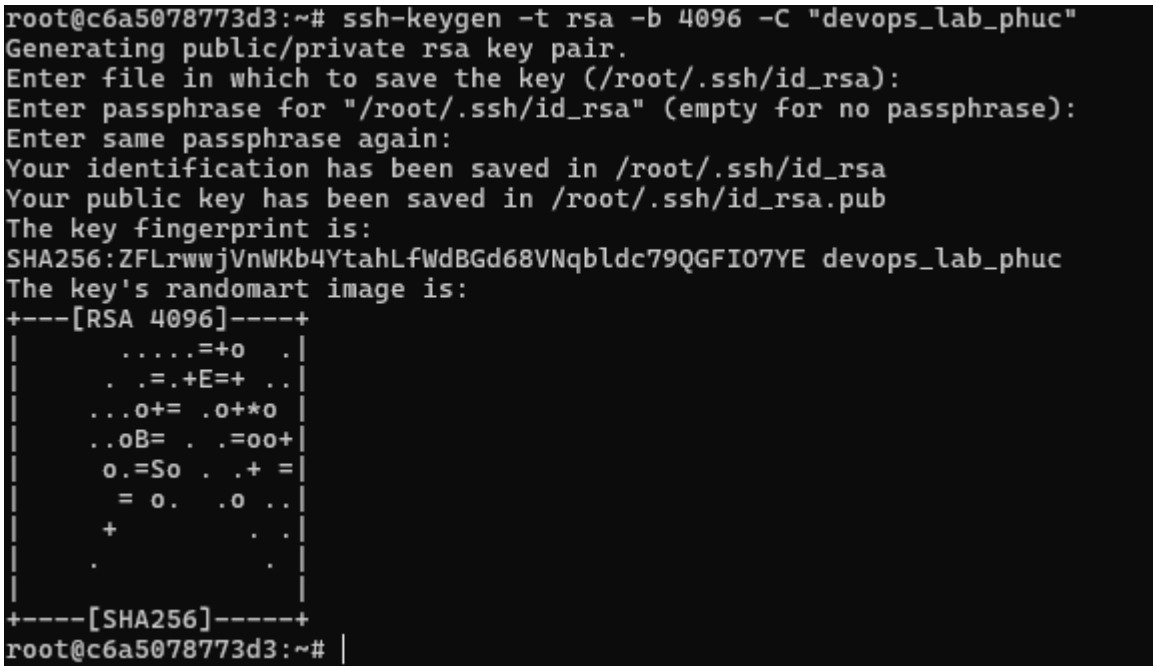
Hình 2.9: Khởi chạy container với tiến trình chính /usr/sbin/sshd -D, dịch vụ SSH duy trì trạng thái hoạt động ổn định

2.5. Thiết lập xác thực bảo mật không dùng mật khẩu bằng SSH Key

Xác thực bằng mật khẩu truyền thống tiềm ẩn nguy cơ bị tấn công dò quét (brute-force) và không thể tự động hóa trong các kịch bản CI/CD. Do đó, việc cấu hình xác thực bằng cặp khóa công khai / bí mật (Asymmetric Cryptography) là yêu cầu bắt buộc.

Sinh viên thực hiện tạo cặp khóa RSA có độ dài an toàn 4096-bit trên máy khách bằng công cụ ssh-keygen:

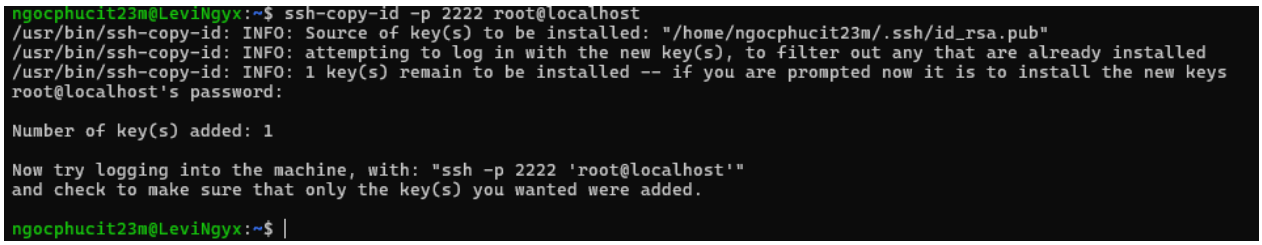
```
ssh-keygen -t rsa -b 4096 -C "devops_lab_phuc"
# Kết quả tạo ra hai tệp: id_rsa (khóa bí mật) và id_rsa.pub (khóa công khai)
```



Hình 2.10: Khởi tạo cặp khóa xác thực RSA với độ dài khóa 4096-bit bằng công cụ ssh-keygen

Sau khi cặp khóa được tạo lập, khóa công khai (id_rsa.pub) được chuyển và cài đặt tự động vào danh sách ủy quyền (~/.ssh/authorized_keys) của container thông qua lệnh chuyên dụng ssh-copy-id:

```
ssh-copy-id -p 2222 root@localhost
ssh -p 2222 root@localhost
# Hệ thống đăng nhập lập tức vào container mà KHÔNG yêu cầu nhập mật khẩu
```



Hình 2.11: Kết quả cài đặt khóa công khai thành công qua lệnh ssh-copy-id và đăng nhập không cần mật khẩu

2.6. Tầm quan trọng của SSH Key trong tự động hóa CI/CD

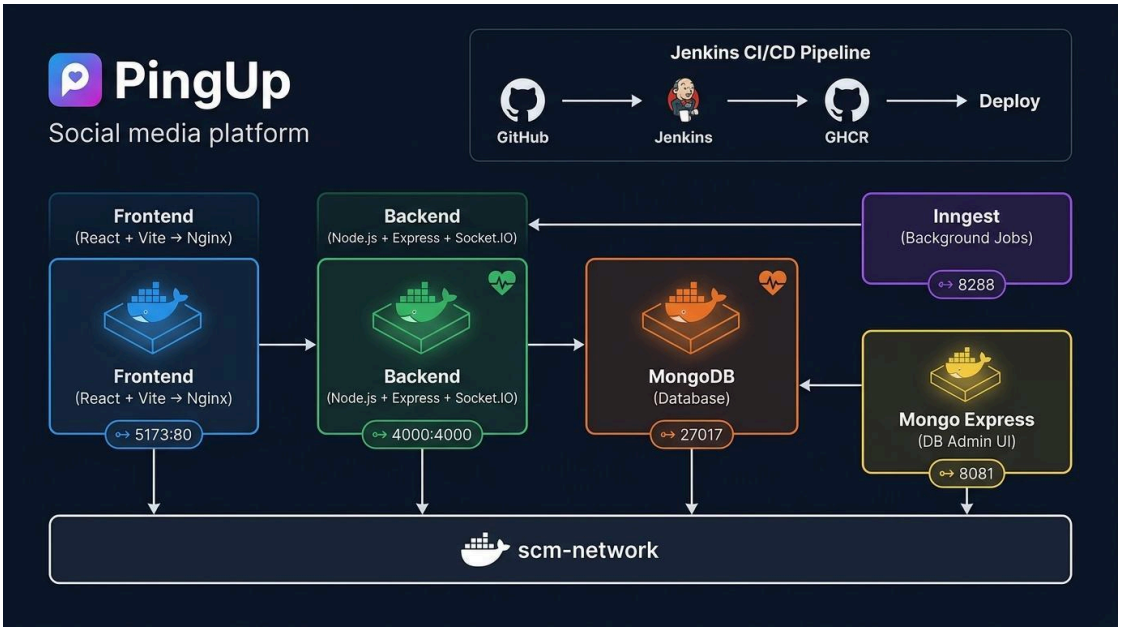
Việc làm chủ kỹ thuật xác thực SSH Key giữ vai trò mang tính sống còn đối với một kỹ sư DevOps:

- **Kéo mã nguồn tự động:**Các máy chủ điều phối CI/CD (như Jenkins, GitLab CI) cần sao chép mã nguồn từ GitHub hoặc GitLab thông qua giao thức Git SSH (git@github.com:...) mà không lưu mật khẩu dạng bản rõ.
- **Triển khai hạ tầng từ xa:**Trong giai đoạn Deploy, server CI/CD sẽ mở kết nối SSH an toàn tới các máy chủ Staging hoặc Production để thực thi lệnh cập nhật container (docker pull, docker compose up -d) hoàn toàn tự động.
- **Bảo mật thông tin đăng nhập:**Khóa bí mật được lưu trữ tập trung và mã hóa trong Jenkins Credentials Manager hoặc GitHub Secrets, ngăn chặn rò rỉ thông tin đăng nhập trong lịch sử commit mã nguồn.

CHƯƠNG 3. THIẾT KẾ KIẾN TRÚC HỆ THỐNG VÀ CƠ SỞ DỮ LIỆU PINGUP

3.1. Sơ đồ kiến trúc tổng thể 5 container trên mạng scm-network

Hệ thống PingUp được xây dựng dựa trên kiến trúc phân tán đa container (Multi-container Architecture). Toàn bộ các dịch vụ được đóng gói và vận hành trong một mạng ảo độc lập mang tên 'scm-network' sử dụng trình điều khiển Docker Bridge. Mô hình này giúp cô lập môi trường mạng của ứng dụng với mạng bên ngoài, đồng thời cho phép các container có thể tự động phân giải tên miền và giao tiếp nội bộ với nhau thông qua cơ chế Docker Internal DNS mà không cần chỉ định địa chỉ IP tĩnh.



Hình 3.1: Sơ đồ kiến trúc tổng thể hệ thống PingUp với 5 containers và luồng điều phối CI/CD

3.2. Phân tích chi tiết chức năng của các service trong hệ thống

Hệ thống bao gồm 5 container dịch vụ chuyên biệt đảm nhận các chức năng cụ thể trong chu trình hoạt động của ứng dụng:

Tên Service	Container Name	Image Base / Build	Cổng công khai	Vai trò và nhiệm vụ kỹ thuật
frontend	scm-frontend	Build từ client/ (Nginx 1.27)	5173:80	Phục vụ ứng dụng React SPA, xử lý định tuyến phía client và nén dữ liệu
backend	scm-backend	Build từ server/ (Node 22)	4000:4000	Cung cấp REST API, Socket.IO WebSockets, xử lý xác thực Clerk và Webhook
mongodb	scm-mongodb	mongo:7.0-official	27017:27017	Cơ sở dữ liệu NoSQL chính, lưu trữ tài liệu với volume ánh xạ mongo_data

inngest	scm-inngest	inngest/inngest:latest	8288:8288	Máy chủ điều phối Background Jobs, quản lý hàng đợi sự kiện và bảng điều khiển UI
mongo-express	scm-mongo-express	mongo-express:latest	8081:8081	Giao diện web trực quan phục vụ công tác quản trị, truy vấn CSDL MongoDB

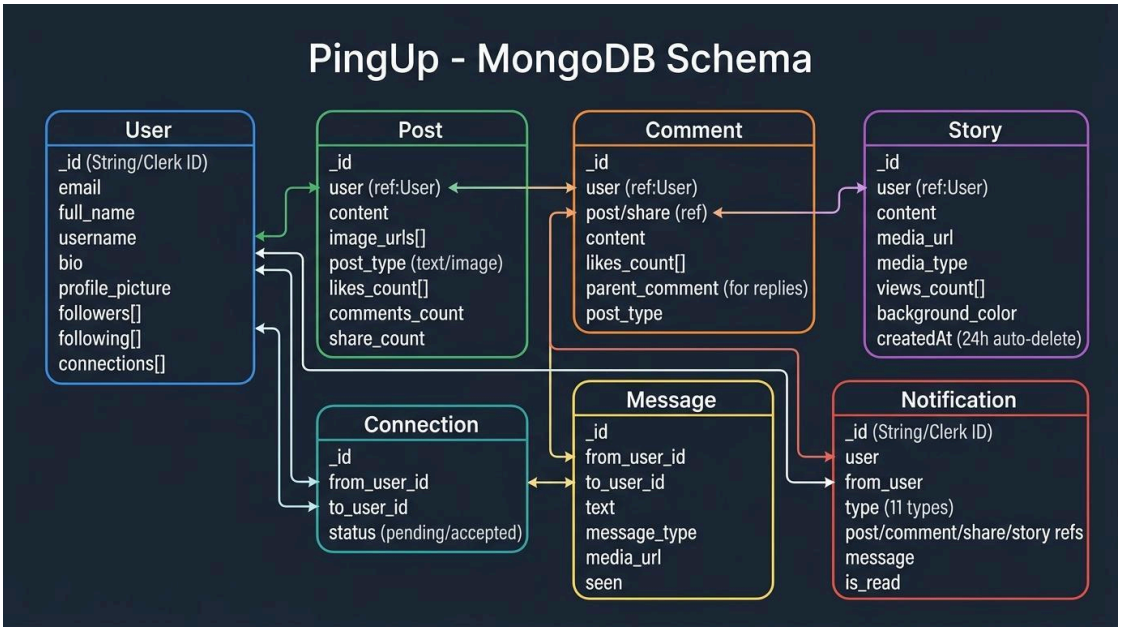
3.3. Luồng dữ liệu và giao tiếp liên dịch vụ

Luồng truyền nhận thông tin trong hệ thống PingUp được tối ưu hóa theo các chuẩn giao thức phù hợp với từng phân hệ:

- **Giao thức tải trang tĩnh:**Trình duyệt người dùng gửi yêu cầu HTTP tới cổng 5173 (container Nginx). Nginx trả về gói JavaScript bundle đã biên dịch sẵn của React. Mọi tương tác chuyển trang trên thanh địa chỉ đều được React Router đón nhận và render tại chỗ nhờ cấu hình try_files.
- **Giao thức RESTful API:**Các tác vụ đọc/ghi dữ liệu bài viết, tải trang cá nhân được client gửi tới backend API (cổng 4000) qua thư viện Axios với định dạng JSON tiêu chuẩn. Backend xác thực người dùng bằng Clerk JWT token đính kèm trong HTTP Header.
- **Kênh giao tiếp thời gian thực:**Để duy trì tính năng trò chuyện và thông báo tức thời, client mở một kênh kết nối song công WebSockets tới backend. Khi có tương tác mới (như thích bài viết hoặc gửi tin nhắn), backend phát sự kiện emit tới đúng phòng 'user_{userId}' của người nhận.
- **Kênh tác vụ nền:**Khi backend tiếp nhận các sự kiện cần xử lý nền hoặc định thời (như đăng story 24h, gửi thư nhắc nhở kết nối), nó kích hoạt sự kiện tới máy chủ Inngest thông qua endpoint /api/inngest. Inngest sẽ đảm bảo tác vụ được hoàn tất độc lập mà không chặn luồng xử lý chính của người dùng.

3.4. Thiết kế cơ sở dữ liệu MongoDB (7 collections)

PingUp thiết kế mô hình dữ liệu gồm 7 collection chính trên MongoDB, được chuẩn hóa và quản lý thông qua thư viện Mongoose ODM. Cấu trúc này đảm bảo tính toàn vẹn dữ liệu, hỗ trợ truy vấn phức tạp và mở rộng linh hoạt.



Hình 3.2: Thiết kế Schema MongoDB với 7 collections và các quan hệ liên kết dữ liệu

3.4.1. Phân tích các đặc điểm thiết kế Schema nổi bật

Thiết kế cơ sở dữ liệu trong dự án chứa đựng nhiều giải pháp kỹ thuật tối ưu hóa chuyên biệt:

- **Sử dụng Clerk String ID làm khóa chính (_id):**Thông thường MongoDB tự sinh khóa chính dạng ObjectId ('507f1f77bcf86cd799439011'). Tuy nhiên, do dự án tích hợp với nền tảng Clerk Authentication, hệ thống đã ghi đè trường '_id' thành chuỗi String chính là Clerk User ID (ví dụ: 'user_2tXyZ...'). Giải pháp này giúp loại bỏ hoàn toàn bảng trung gian ánh xạ mã định danh, cho phép đối chiếu trực tiếp dữ liệu từ JWT Token vào MongoDB mà không tốn chi phí truy vấn thêm.
- **Cấu trúc Post đa phương tiện:**Hỗ trợ phân loại bài viết linh hoạt theo mảng enum: ['text', 'image', 'text_with_image', 'video', 'text_with_video']. Mảng 'likes_count' lưu trữ danh sách các String User ID của những người dùng đã tương tác thích, cho phép kiểm tra trạng thái tương tác với độ phức tạp thuật toán O(1) hoặc truy vấn mảng trực tiếp.
- **Mô hình bình luận lồng nhau (Nested Reply):**Để hỗ trợ tính năng trả lời bình luận nhiều cấp độ mà không làm biến dạng cấu trúc phân cấp, Schema chứa trường 'parent_comment' tham chiếu ngược về chính collection Comment. Nếu 'parent_comment = null', đó là bình luận gốc; nếu có giá trị ObjectId, hệ thống tự động nhận diện đây là phản hồi lồng cấp.
- **Hệ thống thông báo đầy 11 trạng thái:**Quản lý chặt chẽ 11 phân loại sự kiện hệ thống bằng enum nghiêm ngặt: 'like_post', 'like_comment', 'like_share', 'comment_post', 'comment_share', 'reply_comment', 'share_post', 'follow', 'new_post', 'new_story', 'change_profile'. Mỗi thông báo đều có cờ trạng thái 'is_read' và liên kết tới đối tượng gây ra hành động.

3.5. Hệ thống Background Jobs và Durable Functions với Inngest

Một trong những ưu điểm kỹ thuật vượt trội của PingUp là việc ứng dụng Inngest để xử lý các tác vụ nền theo cơ chế 'Durable Functions' (hàm bền bỉ). Thay vì sử dụng các hàng đợi tác vụ truyền thống như Redis/Bull hoặc RabbitMQ (vốn đòi hỏi hạ tầng phức tạp và tiêu hao nhiều RAM), Inngest cho phép lập trình viên định nghĩa các hàm có khả năng tạm dừng trạng thái thực thi trong một khoảng thời gian dài và tự động thức dậy khi đến hạn mà hoàn toàn không giữ kết nối mạng hay tiêu tốn chu kỳ tính toán của CPU.

Function ID	Sự kiện kích hoạt (Trigger)	Cơ chế thực thi & Logic xử lý nghiệp vụ
sync-user-creation	clerk/user.created	Tiếp nhận webhook từ Clerk, tự động khởi tạo bản ghi User mới trong MongoDB (JIT Provisioning)
sync-user-updation	clerk/user.updated	Cập nhật tức thì các thay đổi họ tên, ảnh đại diện, email từ Clerk vào MongoDB
delete-user-from-clerk	clerk/user.deleted	Kích hoạt chuỗi xóa phân tầng (Cascade Delete) toàn bộ bài viết, bình luận, tin nhắn liên quan
send-new-connection-request-reminder	app/connection-request	Gửi email thông báo tức thời qua SMTP Brevo, sau đó sleepUntil 24h nhắc lại nếu chưa chấp nhận
story-delete	app/story.delete	Tạm dừng bền bỉ đúng 24 giờ (step.sleepUntil), tự động kích hoạt xóa story khỏi CSDL khi hết hạn
send-unseen-messages-notification	Cron: 0 9 * * * (9:00 AM)	Định thời mỗi ngày lúc 9h sáng, quét tổng hợp tin nhắn chưa đọc và gửi email nhắc nhở

> **Phân tích chuyên sâu: Tại sao dự án sử dụng step.sleepUntil của Inngest thay vì hàm setTimeout?**
> *Nếu sử dụng hàm setTimeout(fn, 24 * 60 * 60 * 1000) của JavaScript, timer được lưu trữ trong bộ nhớ RAM tạm thời của tiến trình Node.js. Khi máy chủ khởi động lại, nâng cấp phiên bản hoặc container bị tắt, toàn bộ các timer này sẽ biến mất vĩnh viễn, dẫn đến việc các Story cũ không bao giờ bị xóa và rác dữ liệu tích tụ trong cơ sở dữ liệu.*

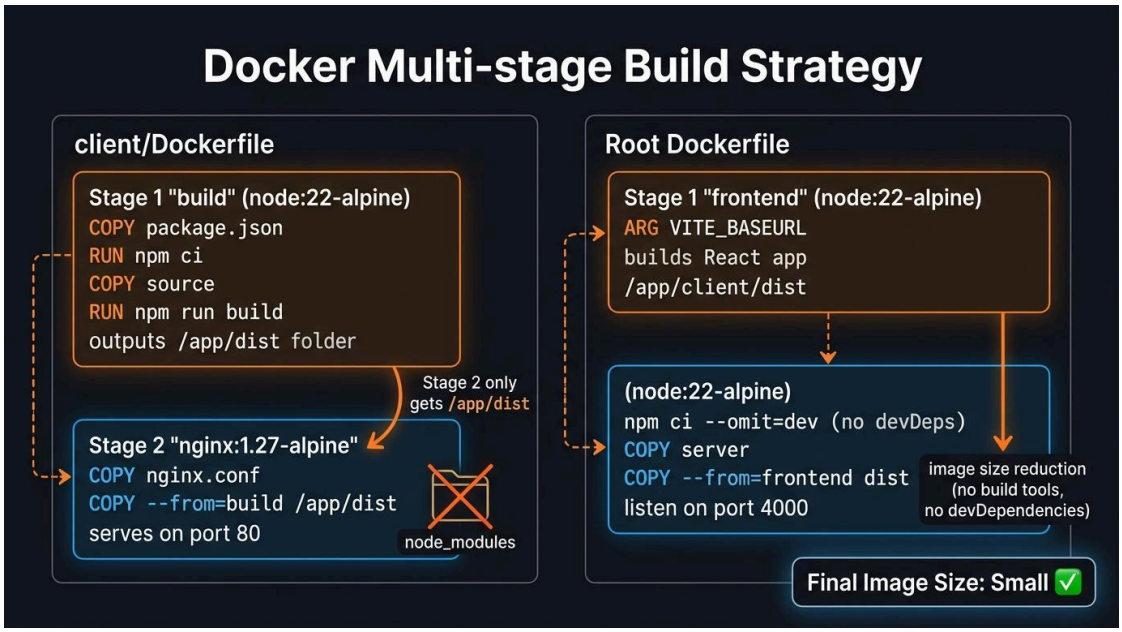
> *Ngược lại, với Inngest Durable Functions, trạng thái của bước thực thi được lưu trữ bền bỉ trong cơ sở dữ liệu của Inngest Server. Dù container backend có bị tắt trong nhiều giờ thì khi được khởi động lại, Inngest vẫn xác định chính xác thời điểm hết hạn và gọi hàm xóa Story đúng như kế hoạch đã định. Đây là kiến trúc chuẩn mực được ứng dụng tại các hệ thống quy mô lớn.*

CHƯƠNG 4. ĐÓNG GÓI VÀ TỐI ƯU HÓA VỚI DOCKER & DOCKER COMPOSE

4.1. Chiến lược Docker Multi-stage Build chuyên sâu

Kỹ thuật Multi-stage Build trong Docker cho phép chia quá trình xây dựng image thành nhiều giai đoạn kế tiếp nhau trong cùng một Dockerfile. Mục tiêu cốt lõi của kỹ thuật này là tách biệt hoàn toàn môi trường biên dịch (chứa mã nguồn, trình biên dịch, công cụ đóng gói cồng kềnh) khỏi môi trường vận hành sản xuất (chỉ chứa các tệp nhị phân hoặc mã tĩnh cần thiết tối thiểu).

Dự án PingUp áp dụng hai chiến lược Multi-stage Build riêng biệt tùy thuộc vào mô hình triển khai:



Hình 4.1: Sơ đồ đối chiếu hai chiến lược Docker Multi-stage Build được triển khai trong dự án

4.2. Phân tích Dockerfile Frontend (client/Dockerfile)

Tệp cấu hình client/Dockerfile triển khai mô hình chuyển đổi từ môi trường biên dịch Node.js sang máy chủ phục vụ tĩnh siêu nhẹ Nginx Alpine:

```
# ===== GIAI ĐOẠN 1: Biên dịch mã nguồn React với Vite =====
FROM node:22-alpine AS build
```

```
WORKDIR /app

# Khai báo các tham số biên dịch build-time
ARG VITE_BASEURL=http://localhost:4000
ARG VITE_CLERK_PUBLISHABLE_KEY=pk_test_...
ENV VITE_BASEURL=$VITE_BASEURL
ENV VITE_CLERK_PUBLISHABLE_KEY=$VITE_CLERK_PUBLISHABLE_KEY

# Kỹ thuật Layer Caching: sao chép tệp định nghĩa thư viện trước
COPY package.json package-lock.json ./
RUN npm ci

# Sao chép toàn bộ mã nguồn và thực thi đóng gói
COPY . .
RUN npm run build   # Thư mục đích: /app/dist

# ===== GIAI ĐOẠN 2: Môi trường vận hành sản xuất siêu nhẹ Nginx =====
FROM nginx:1.27-alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

Một điểm nhấn kỹ thuật quan trọng trong tệp nginx.conf của dự án là việc cấu hình quy tắc định tuyến cho ứng dụng React Router:

```
server {
    listen 80;
    server_name localhost;

    # Nén Gzip nhằm giảm thiểu băng thông mạng
    gzip on;
    gzip_types text/plain text/css application/json application/javascript text/xml;

    location / {
        root /usr/share/nginx/html;
        index index.html index.htm;
        # Định tuyến SPA fallback: điều hướng mọi đường dẫn về index.html
        try_files $uri $uri/ /index.html;
    }
}
```

Ý nghĩa kỹ thuật của chỉ thị 'try_files $uri $uri/ /index.html':
React là một ứng dụng Single Page Application (SPA). Mọi thao tác chuyển trang trên trình duyệt (như /profile, /chat) là các đường dẫn ảo được điều khiển bằng JavaScript ở phía client, không tồn tại các tệp vật lý tương ứng trên ổ cứng máy chủ Nginx. Nếu thiếu cấu hình này, khi người dùng tải lại trang tại đường dẫn /profile, Nginx sẽ trả về mã lỗi 404 Not Found. Với try_files, Nginx sẽ ưu tiên tìm tệp tĩnh thật; nếu không thấy sẽ tự động trả về tệp index.html để React Router khởi chạy và hiển thị đúng trang.

4.3. Phân tích Dockerfile Backend (server/Dockerfile)

Phía backend Express.js được đóng gói trên nền tảng image chính thức Node.js 22 bản Alpine rút gọn. Tệp server/Dockerfile áp dụng lệnh 'npm ci --omit=dev' để loại bỏ hoàn toàn các gói phụ thuộc dùng cho phát triển (như nodemon, eslint), giúp kích thước image giảm đáng kể và giảm thiểu các lỗ hổng bảo mật tiềm ẩn.

```
FROM node:22-alpine
WORKDIR /app

# Tối ưu hóa bộ nhớ đệm Docker layer
COPY package.json package-lock.json ./
RUN npm ci --omit=dev

COPY . .

ENV NODE_ENV=production
ENV PORT=4000

EXPOSE 4000
CMD ["node", "server.js"]
```

4.4. Quản lý đa dịch vụ với Docker Compose và cơ chế Healthcheck

Trong các hệ thống phân tán, vấn đề 'cuộc đua khởi động' (Race Condition) rất thường xuyên xảy ra khi các dịch vụ phụ thuộc (như Backend) khởi chạy trước khi dịch vụ cốt lõi (như Database) hoàn tất quá trình nạp dữ liệu. Chỉ thị 'depends_on' thông thường chỉ kiểm tra việc container đã được kích hoạt (started) chứ không biết dịch vụ bên trong đã sẵn sàng (ready) hay chưa.

Để giải quyết triệt để vấn đề này, docker-compose.yml của dự án triển khai cơ chế Healthcheck chủ động:

```
mongodb:
    image: mongo:7
    container_name: scm-mongodb
    ports:
      - "27017:27017"
    volumes:
      - mongo_data:/data/db
    healthcheck:
      test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping')"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s

  backend:
    build: ./server
    container_name: scm-backend
    init: true
```

```
    depends_on:
      mongodb:
        condition: service_healthy   # BẮT BUỘC MongoDB phải trả về Healthy mới được
start
    healthcheck:
      test: ["CMD", "node", "-e", "fetch('http://127.0.0.1:4000/').then(r =>
process.exit(r.ok ? 0 : 1)).catch(() => process.exit(1))"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 15s
```

4.5. Phân tích tầm quan trọng của cờ init: true chống zombie process

Trong tệp docker-compose.yml, dịch vụ backend được khai báo thêm thuộc tính đặc biệt 'init: true':

Giải thích chuyên sâu về 'init: true' và tiến trình PID 1 trong container:
Trong nhân hệ điều hành Linux, tiến trình mang PID 1 (thường là systemd hoặc init) có hai nhiệm vụ tối quan trọng: (1) Chuyển tiếp các tín hiệu điều khiển hệ thống (như SIGTERM, SIGINT) đến toàn bộ các tiến trình con; và (2) Thu dọn các tiến trình con đã kết thúc nhưng chưa được giải phóng tài nguyên (gọi là Zombie/Defunct processes).
Khi chạy ứng dụng Node.js trong Docker container mà không có init, tiến trình node sẽ trực tiếp giữ PID 1. Tuy nhiên, Node.js không được thiết kế để làm init system, nó không tự động dọn dẹp tiến trình con và thường bỏ qua tín hiệu SIGTERM. Hệ quả là khi thực hiện 'docker compose down', lệnh sẽ bị treo đơ trong 10 giây trước khi Docker buộc phải cưỡng chế kill container bằng SIGKILL.
Bằng cách thêm 'init: true', Docker sẽ tự động chèn một tiến trình init siêu nhỏ (Tini) làm PID 1, giúp chuyển tiếp tín hiệu chuẩn xác, cho phép Node.js thực thi Graceful Shutdown ngay lập tức mà không có độ trễ.

4.6. Cấu hình triển khai Production với docker-compose.prod.yml

Để phục vụ triển khai môi trường sản xuất thông qua pipeline CI/CD, dự án sử dụng tệp ghi đè docker-compose.prod.yml:

```
# docker-compose.prod.yml: Kỹ thuật ghi đè cấu hình sản xuất
services:
  frontend:
    build: !reset null             # Hủy bỏ hoàn toàn cấu hình build cục bộ
    image: ${FRONTEND_IMAGE_TAG}   # Kéo image đã được đóng gói từ GHCR
    container_name: !reset null

  backend:
    build: !reset null
    image: ${BACKEND_IMAGE_TAG}
    container_name: !reset null
```

Cú pháp '!reset null' là tính năng nâng cao của Docker Compose v2 Specification. Nó cho phép hủy bỏ thuộc tính 'build' đã khai báo trong tệp gốc, buộc Docker Compose phải nạp các image đã được gắn thẻ theo Git SHA thông qua hai biến môi trường ${FRONTEND_IMAGE_TAG} và ${BACKEND_IMAGE_TAG}.

CHƯƠNG 5. XÂY DỰNG PIPELINE CI/CD TỰ ĐỘNG VỚI JENKINS

5.1. Mô hình Docker-out-of-Docker (DooD) và phân tích so sánh với DinD

Để xây dựng một máy chủ tự động hóa Jenkins có khả năng biên dịch và triển khai các container khác, kỹ sư DevOps có thể lựa chọn một trong hai kiến trúc: Docker-in-Docker (DinD) hoặc Docker-out-of-Docker (DooD). Dự án PingUp quyết định lựa chọn giải pháp DooD bằng cách mount socket của Docker máy chủ (/var/run/docker.sock) trực tiếp vào bên trong container Jenkins.

Tiêu chí kỹ thuật	Mô hình DooD (Áp dụng trong đề tài)	Mô hình DinD (Docker-in-Docker)
Cơ chế hoạt động	Dùng chung Docker daemon của máy Host qua socket /var/run/docker.sock	Khởi chạy một daemon Docker độc lập hoàn toàn bên trong container
Chia sẻ bộ nhớ đệm (Cache)	Chia sẻ trực tiếp image cache với Host, tốc độ build cực nhanh	Cache bị cô lập riêng biệt, mất toàn bộ cache khi container bị tạo lại
Yêu cầu đặc quyền (Security)	Chỉ cần mount socket và cấp quyền nhóm docker, không cần privileged	Bắt buộc phải bật cờ nguy hiểm '--privileged', tiềm ẩn rủi ro chiếm quyền root
Hiệu năng và tài nguyên	Hiệu năng đạt mức tối đa do không có lớp ảo hóa trung gian	Tiêu tốn nhiều RAM/CPU do phải chạy lồng hai tầng daemon Docker
Mục đích ứng dụng phù hợp	Triển khai máy chủ CI/CD nội bộ, phòng lab trường học và môi trường staging	Môi trường điện toán đám mây đa người dùng (Multi-tenant) cô lập tuyệt đối

5.2. Cấu hình môi trường Jenkins và bảo mật Credentials GHCR

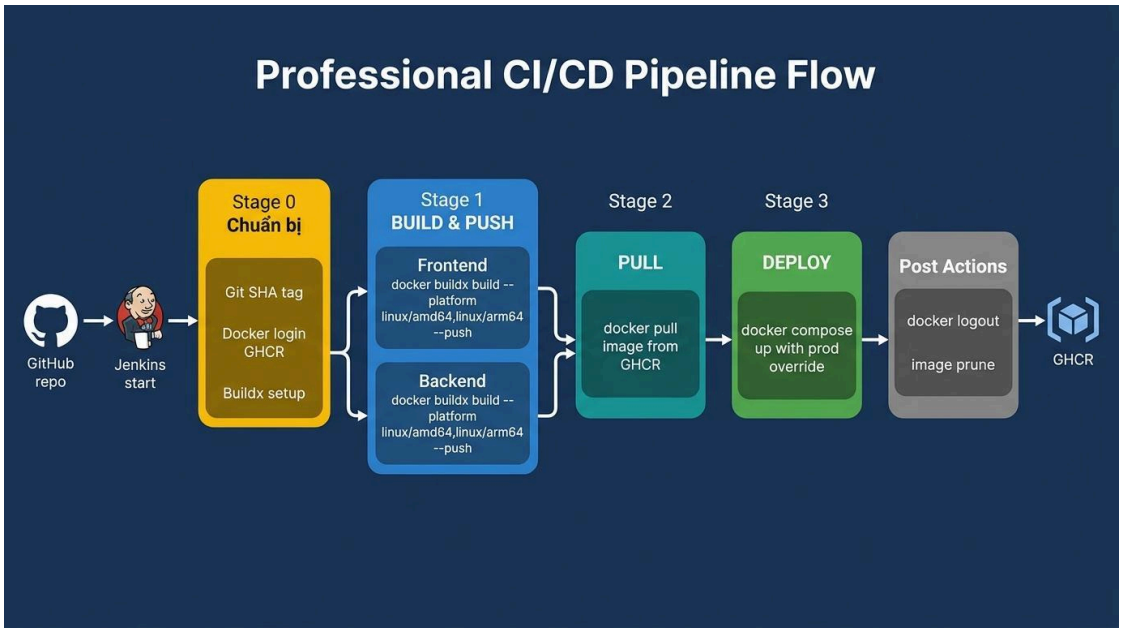
Container Jenkins được khởi tạo kèm theo volume dữ liệu 'jenkins_home' và socket Docker máy chủ:

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins/jenkins:lts
```

Nhằm tuân thủ nguyên tắc an toàn thông tin, các thông tin định danh truy cập GitHub Container Registry (GHCR) tuyệt đối không được ghi dạng bản rõ vào mã nguồn Jenkinsfile. Thay vào đó, sinh viên cấu hình trong kho quản lý bảo mật của Jenkins (Manage Jenkins → Credentials → Global) với ID 'ghcr-credentials'. Khi pipeline khởi chạy, Jenkins sẽ tự động inject an toàn hai biến môi trường tạm thời: GHCR_CREDS_USR (tên tài khoản) và GHCR_CREDS_PSW (Personal Access Token có quyền ghi packages).

5.3. Phân tích chi tiết Declarative Jenkinsfile qua từng giai đoạn

Quy trình tự động hóa của Jenkins được định nghĩa thông qua tệp kịch bản Declarative Jenkinsfile bao gồm 4 giai đoạn chính thức:



Hình 5.1: Sơ đồ quy trình thực thi Pipeline CI/CD 4 giai đoạn trên máy chủ Jenkins

5.3.1. Giai đoạn 'Chuẩn bị' (Environment Initialization)

Trong giai đoạn mở đầu, kịch bản trích xuất 7 ký tự mã băm đầu tiên của Git Commit hiện tại bằng lệnh 'git rev-parse --short=7 HEAD'. Mã băm này đóng vai trò là một định danh duy nhất (Unique Immutable Tag) cho phiên bản image, đảm bảo tính truy vết nguồn gốc mã nguồn.

```
stage('Chuẩn bị') {
  steps {
    script {
      env.SHA_SHORT = sh(script: 'git rev-parse --short=7 HEAD', returnStdout:
true).trim()
      env.FRONTEND_IMAGE = "${REGISTRY}/${GITHUB_USER}/scm-frontend:${env.SHA_SHORT}"
```

```
      env.FRONTEND_LATEST = "${REGISTRY}/${GITHUB_USER}/scm-frontend:latest"
      env.BACKEND_IMAGE = "${REGISTRY}/${GITHUB_USER}/scm-backend:${env.SHA_SHORT}"
      env.BACKEND_LATEST = "${REGISTRY}/${GITHUB_USER}/scm-backend:latest"
    }
    sh '''
      echo "$GHCR_CREDS_PSW" | docker login ${REGISTRY} -u "$GHCR_CREDS_USR"
--password-stdin
      docker buildx create --name multiarch --driver docker-container --use 2>/dev/null
|| docker buildx use multiarch
      docker buildx inspect --bootstrap
    '''
  }
}
```

5.3.2. Giai đoạn 1: 'BUILD & PUSH' song song (Parallel Execution)

Nhằm rút ngắn thời gian đóng gói, Jenkinsfile kích hoạt khối lệnh 'parallel' cho phép xây dựng đồng thời cả hai image Frontend và Backend. Công cụ Docker Buildx được chỉ định biên dịch hỗ trợ đa kiến trúc CPU: 'linux/amd64,linux/arm64', cho phép hình ảnh container có thể hoạt động mượt mà trên cả các máy chủ x86 truyền thống lẫn các máy chủ ARM hiện đại (như Apple Silicon M-series hoặc AWS Graviton).

5.3.3. Giai đoạn 2 & 3: 'PULL' và 'DEPLOY'

Sau khi các image mới được xuất bản lên GHCR thành công, giai đoạn PULL sẽ tiến hành kéo các image này về máy chủ triển khai. Tiếp theo, giai đoạn DEPLOY nạp các biến môi trường thẻ phiên bản và thực thi tái khởi động hệ thống:

```
stage('3. DEPLOY') {
  steps {
    withEnv([
      "FRONTEND_IMAGE_TAG=${env.FRONTEND_IMAGE}",
      "BACKEND_IMAGE_TAG=${env.BACKEND_IMAGE}"
    ]) {
      sh '''
        mkdir -p ./server && cp ./server/.env.example ./server/.env 2>/dev/null ||
touch ./server/.env
        docker compose -p ${DEPLOY_PROJECT} \
          -f docker-compose.yml -f docker-compose.prod.yml \
          up -d --remove-orphans
      '''
    }
  }
}
```

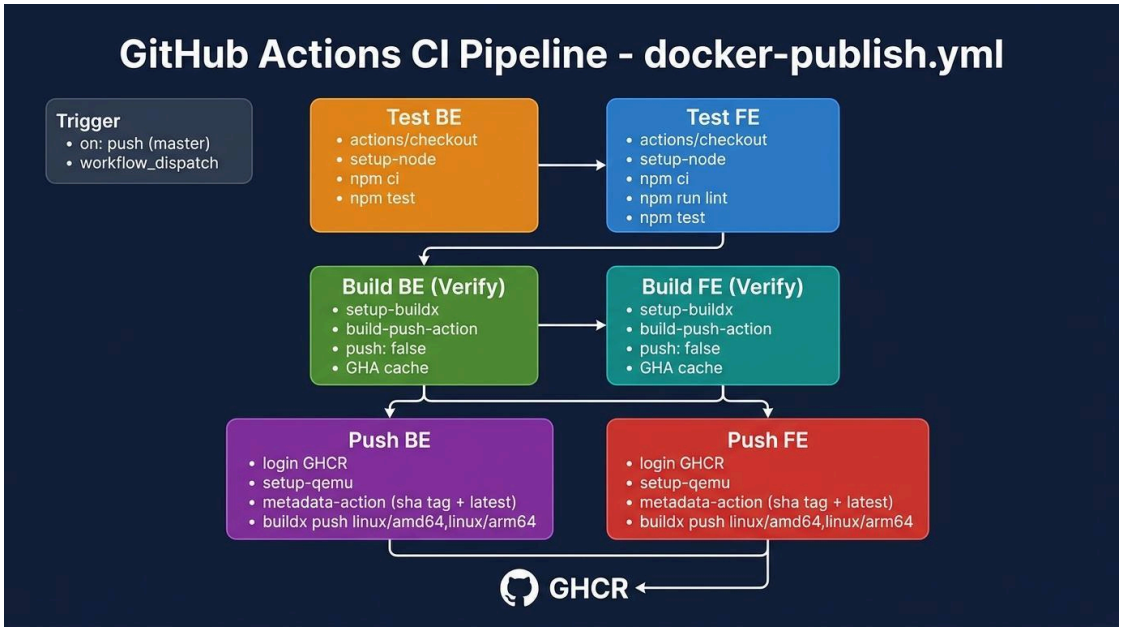
5.3.4. Khối xử lý dọn dẹp sau triển khai (Post Actions)

Khối lệnh 'post { always { ... } }' luôn được kích hoạt để thực hiện bảo mật bằng cách đăng xuất tài khoản GHCR ('docker logout'), đồng thời dọn dẹp các image tạm vô danh (dangling images) có tuổi thọ trên 7 ngày bằng lệnh 'docker image prune' nhằm giải phóng dung lượng đĩa cứng.

CHƯƠNG 6. TỰ ĐỘNG HÓA CI TRÊN CLOUD VỚI GITHUB ACTIONS

6.1. Kiến trúc workflow GitHub Actions (.github/workflows)

Bên cạnh hệ thống Jenkins tự vận hành (Self-hosted), dự án PingUp còn tích hợp quy trình Tích hợp liên tục (CI) ngay trên nền tảng đám mây của GitHub thông qua tệp cấu hình '.github/workflows/docker-publish.yml'. Quy trình này tự động kích hoạt mỗi khi có sự kiện đẩy mã nguồn (Push) lên nhánh master hoặc thông qua kích hoạt thủ công (workflow_dispatch).



Hình 6.1: Sơ đồ kiến trúc các Jobs kiểm thử và đóng gói đa kiến trúc trong GitHub Actions

6.2. Phân tích 6 jobs: Kiểm thử, Kiểm tra build và Đẩy Image

Quy trình làm việc được chia thành 6 jobs chuyên biệt, thực thi song song trên các máy ảo Ubuntu độc lập (ubuntu-latest):

- **Job test-backend (Test BE):**Thiết lập môi trường Node.js 20, tự động tải các gói phụ thuộc qua npm ci và thực thi các bài kiểm thử đơn vị của máy chủ API.
- **Job build-backend (Build BE):**Khởi chạy Docker Buildx để kiểm tra tính hợp lệ của server/Dockerfile (với cờ push: false). Bước này đảm bảo mã nguồn biên dịch hoàn toàn thành công trước khi xuất bản chính thức.
- **Job push-backend (Push BE):**Phụ thuộc vào kết quả của 2 job trên (needs: [test-backend, build-backend]). Sử dụng action QEMU và Buildx để đóng gói đa kiến trúc (AMD64 + ARM64) và xuất bản lên kho lưu trữ ghcr.io.
- **Job test-frontend (Test FE):**Kiểm tra cú pháp giao diện bằng lệnh npm run lint và chạy kiểm thử tự động của ứng dụng React Vite.
- **Job build-frontend (Build FE):**Kiểm tra tính hợp lệ của client/Dockerfile và các tham số môi trường biên dịch VITE_*.

- **Job push-frontend (Push FE):**Xuất bản image giao diện chính thức lên GHCR kèm các nhãn metadata tự động (nhánh, commit SHA và thẻ latest).

6.3. Kỹ thuật tối ưu: Path Filtering, GHA Cache và GITHUB_TOKEN

Quy trình GitHub Actions trong dự án áp dụng ba kỹ thuật tối ưu hóa tiên tiến nhất hiện nay:

- **Lọc đường dẫn thay đổi (Path Filtering):**Sử dụng bộ lọc đường dẫn 'paths: [client/**, server/**, .github/workflows/**]'. Nếu lập trình viên chỉ cập nhật tệp tài liệu README.md hoặc tệp báo cáo Word, pipeline sẽ tự động bỏ qua (skip), giúp tiết kiệm tối đa hạn ngạch thời gian chạy miễn phí (2000 phút/tháng) của tài khoản GitHub.
- **Bộ nhớ đệm chuyên dụng (GitHub Actions Cache):**Tích hợp 'cache-from: type=gha,scope=backend' và 'cache-to: type=gha,mode=max'. Docker Buildx sẽ lưu lại các layer đã biên dịch lên hạ tầng bộ nhớ đệm đám mây của GitHub Actions. Ở các lần commit tiếp theo, các layer không thay đổi (như bước cài đặt npm) được phục hồi ngay lập tức, giúp rút ngắn thời gian build từ 4 phút xuống chỉ còn dưới 45 giây.
- **Xác thực tự động không khóa tĩnh:**Sử dụng mã định danh bảo mật tạm thời '${{ secrets.GITHUB_TOKEN }}' được cấp phát tự động cho mỗi phiên chạy. Lập trình viên không cần phải tạo hay quản lý các mã PAT tĩnh có nguy cơ bị lộ lọt, đồng thời tuân thủ chặt chẽ nguyên tắc bảo mật thông qua việc cấp quyền hạn chế 'packages: write'.

6.4. Bảng so sánh toàn diện giữa Jenkins và GitHub Actions

Bảng 6.1 dưới đây phân tích so sánh toàn diện vai trò của hai nền tảng CI/CD được triển khai trong đồ án:

Đặc tính kỹ thuật	Hệ thống Jenkins (Self-hosted)	Hệ thống GitHub Actions (Cloud)
Môi trường thực thi	Máy chủ nội bộ (DooD mount /var/run/docker.sock)	Hạ tầng đám mây máy ảo dùng một lần (Ephemeral VM ubuntu-latest)
Vai trò cốt lõi trong dự án	Đảm nhận chính khâu Triển khai liên tục (CD) cục bộ	Đảm nhận chính khâu Tích hợp liên tục (CI) trên Cloud
Cơ chế xác thực Registry	Sử dụng Personal Access Token tĩnh lưu trong Jenkins Credentials	Sử dụng GITHUB_TOKEN tự động sinh mới theo từng phiên chạy
Cơ chế bộ nhớ đệm (Cache)	Tận dụng trực tiếp Docker layer cache có sẵn trên máy chủ Host	Sử dụng dịch vụ GitHub Actions Cache Service (type=gha)

Chi phí và hạ tầng	Hoàn toàn miễn phí, phụ thuộc vào năng lực phần cứng máy chủ	Miễn phí 2000 phút/tháng cho các kho mã nguồn công khai/cá nhân
Khả năng triển khai nội bộ	Cực kỳ mạnh mẽ, tương tác trực tiếp với Docker socket máy chủ	Phức tạp hơn, cần thiết lập Self-hosted Runner hoặc mở kết nối SSH

CHƯƠNG 7. HƯỚNG DẪN CÀI ĐẶT, VẬN HÀNH VÀ KỊCH BẢN KIỂM THỬ (DEMO)

7.1. Yêu cầu môi trường phần cứng và phần mềm máy trạm

Để khởi chạy và kiểm thử toàn diện hệ thống PingUp phục vụ buổi vấn đáp trực tiếp tại lớp học, máy trạm của sinh viên cần đáp ứng các điều kiện tiêu chuẩn sau:

- **Cấu hình phần cứng:**Bộ vi xử lý (CPU) tối thiểu 4 nhân, bộ nhớ trong (RAM) tối thiểu 8 GB (khuyến nghị 16 GB), ổ cứng trống tối thiểu 10 GB.
- **Hệ điều hành:**Hệ điều hành Windows 10/11 64-bit có bật tính năng WSL2, hoặc Ubuntu 22.04/24.04 LTS.
- **Phần mềm bắt buộc:**Docker Engine phiên bản 24.0 trở lên, Docker Compose v2 (phiên bản 2.20 trở lên), Git CLI và trình duyệt web Chrome/Firefox.

7.2. Hướng dẫn từng bước khởi chạy toàn bộ hệ thống từ mã nguồn

Sinh viên thực hiện tuần tự theo các bước chuẩn hóa sau trên cửa sổ dòng lệnh PowerShell hoặc Bash:

- **Bước 1 - Clone mã nguồn:**Sao chép kho mã nguồn dự án từ GitHub về máy tính cá nhân:

```
git clone https://github.com/nhan-trinh/Devops-26-03.git
cd Devops-26-03
```

- **Bước 2 - Thiết lập biến môi trường:**Khởi tạo tệp biến môi trường từ mẫu định nghĩa có sẵn:

```
cp ./server/.env.example ./server/.env
```

- **Bước 3 - Khởi chạy Docker Compose:**Khởi chạy cụm 5 containers ở chế độ chạy nền:

```
docker compose up -d --build
```

- **Bước 4 - Kiểm tra trạng thái:**Giám sát trạng thái hoạt động của toàn bộ dịch vụ và xác nhận trạng thái Healthy:

```
docker compose ps
```

Kết quả hiển thị trên màn hình dòng lệnh phải xác nhận các container 'scm-mongodb' và 'scm-backend' đều đạt trạng thái '(healthy)' thì hệ thống mới được coi là sẵn sàng tiếp nhận yêu cầu.

7.3. Kịch bản kiểm thử các dịch vụ thực tế và đường dẫn truy cập

Bảng 7.1 tổng hợp các địa chỉ dịch vụ và kịch bản tương tác thực tế phục vụ buổi trình diễn (demo) trước giảng viên:

Dịch vụ kiểm thử	Địa chỉ URL truy cập	Cổng	Kịch bản kiểm tra chức năng (Demo Test Case)
Giao diện chính PingUp	http://localhost:5173	5173	Mở trình duyệt, thực hiện đăng nhập qua Clerk, tạo bài viết mới kèm ảnh và đăng một Story 24h
Backend REST API	http://localhost:4000/	4000	Kiểm tra endpoint sức khỏe, xác nhận phản hồi JSON trạng thái hoạt động và kết nối Socket.IO
Bảng điều khiển Inngest	http://localhost:8288	8288	Xem danh sách 6 functions đã nạp, kiểm tra lịch sử thực thi của hàm story-delete và cronjob
Quản trị CSDL Mongo Express	http://localhost:8081	8081	Đăng nhập giao diện GUI, duyệt danh sách 7 collections và kiểm tra bản ghi người dùng mới tạo

7.4. Hướng dẫn xử lý các sự cố thường gặp (Troubleshooting Guide)

Dưới đây là cẩm nang xử lý nhanh các tình huống lỗi thường phát sinh trong quá trình vận hành hệ sinh thái Docker và CI/CD:

- **Lỗi quyền truy cập Docker Socket:**Hiện tượng: Cửa sổ console báo lỗi 'permission denied while trying to connect to the Docker daemon socket'.

  Nguyên nhân & Khắc phục: Do người dùng chạy Jenkins không thuộc nhóm quyền docker.

Xử lý bằng lệnh: 'docker exec -u root jenkins usermod -aG docker jenkins' rồi khởi động lại container Jenkins.

- **Lỗi rào cản CORS giữa Frontend và Backend:**Hiện tượng: Trình duyệt báo lỗi đỏ 'Cross-Origin Request Blocked' khi ứng dụng React gửi yêu cầu API.
Nguyên nhân & Khắc phục: Biến môi trường FRONTEND_URL trong backend chưa khớp với cổng của client. Kiểm tra tệp server/.env và đảm bảo giá trị cấu hình là 'FRONTEND_URL=http://localhost:5173'.
- **Lỗi đồng bộ Webhook người dùng mới:**Hiện tượng: Đăng nhập thành công trên Clerk nhưng tài khoản không xuất hiện trong cơ sở dữ liệu MongoDB.
Nguyên nhân & Khắc phục: Container Inngest chưa được kết nối với backend. Kiểm tra logs bằng lệnh 'docker compose logs inngest' để xác nhận Inngest đã đăng ký thành công endpoint 'http://backend:4000/api/inngest'.

TÀI LIỆU THAM KHẢO

Các tài liệu học thuật, giáo trình chuyên ngành và tài liệu kỹ thuật chính thức được trích dẫn trong bài báo cáo:

- [1] Ths. Vũ Anh Tuấn, 'Tài liệu hướng dẫn thực hành học phần DevOps', Khoa Công nghệ Thông tin, Trường Đại học Đông Á, 2026.
- [2] Docker Documentation, 'Docker Multi-stage Builds and Layer Caching Best Practices', https://docs.docker.com/build/building/multi-stage/, 2026.
- [3] Docker Documentation, 'Docker Compose Specification v2 & Service Healthcheck', https://docs.docker.com/compose/compose-file/, 2026.
- [4] Jenkins Community, 'Pipeline as Code with Declarative Jenkinsfile', https://www.jenkins.io/doc/book/pipeline/, 2026.
- [5] GitHub Documentation, 'Building and testing Node.js with GitHub Actions & GHCR', https://docs.github.com/en/actions, 2026.
- [6] Inngest Documentation, 'Durable Execution Engine and Background Jobs Architecture', https://www.inngest.com/docs, 2026.
- [7] OpenSSH Manual, 'OpenSSH Server Configuration, SSH Key Generation and Authentication', https://www.openssh.com/manual.html, 2026.
- [8] Mongoose Documentation, 'Mongoose v8 Models, Schemas and Subdocuments Reference', https://mongoosejs.com/docs/, 2026.